

HUD — Build Notes

Open HUD Challenge submission · High-Signal Personal News Feed

Author: Arnob Rizwan · Stack: Next.js 16 · Convex · TypeScript · Tailwind v4 · Vercel

What I built

A heads-up display for the internet: an **auto-scrolling, ranked news feed** that pulls from HackerNews, AI newsletters (TLDR AI, AI News/smol.ai, The Rundown), subreddits, and X; **dedupes** syndicated copies into events; **ranks** each item with a transparent score that **blends your focus areas with what's breaking**; and streams it as a sci-fi HUD. It ships with bookmarks that **resurface**, ****bring-your-own AI key summaries, multi-user accounts, and a live pipeline-operator dashboard**** for tuning ranking and watching evaluation metrics.

It delivers **both** challenge tracks in one app:

- **Developer track:** the fully-functional end-user HUD, deployed to Vercel.
- **AI-engineer track:** a modular, reusable AI/ML pipeline + a management dashboard, documented in [PIPELINE.md](#).

Design decisions (and why)

- **Self-contained Next.js + Convex over the existing Python microservices.** The brief mandates React/Next.js on Vercel. Rather than operate 16 Python services, I rebuilt the pipeline as **pure-TypeScript stages** orchestrated by Convex actions — Vercel-native, one deploy, no cross-service latency. The Python repo remains as the AI-engineer reference.
- **Convex for state + realtime.** Reactive queries give the HUD live updates with zero polling; one system covers DB, auth, scheduled functions (cron ingestion), and server functions. Strong multi-user isolation is enforced in every function.
- **Read-time ranking.** User scores are computed when the feed is read, so changing a focus topic, the focus!"trending mix, or a ranking weight reflect~~s~~**instantly** — no re-batch. Batch-global features (popularity, velocity) are precomputed once.
- **Transparency by default.** Every card exposes its score breakdown; the dashboard exposes the weights and lets operators hot-reload them. Ranking isn't a black box.
- **Graceful AI.** Summaries are extractive by default (free, no key). A BYO key upgrades to grounded abstractive summaries with guardrails; nothing breaks without one.
- **Auto-scroll tuning.** Speed is user-controlled (px/sec), pauses on hover/focus, and supports **J/K/space**. A duplicated list gives a seamless loop.
- **Popularity metric.** Normalized engagement (HN points / Reddit ups / X likes, log-compressed + min-max) blended with cross-source velocity and a source-weight prior.

How it's structured

```
lib/pipeline/  pure stages: ingest · normalize · enrich · dedup(MinHash+LSH) · rank ·
summarize
convex/        schema · auth · pipeline orchestrator · crons · feed · feedback · bookmarks ·
eval · dashboard · notifications
app/ + components/hud/ landing · auth · feed · bookmarks · settings · dashboard (HUD
design system)
```

Pipeline contracts live in [lib/pipeline/types.ts](#) so stages stay decoupled from the app — the challenge's modularity/reusability requirement.

What an employer should know

- **End-to-end ownership:** data ingestion, NLP-ish enrichment, near-duplicate

clustering (MinHash/LSH + union-find), learning-to-rank-style scoring, evaluation harness (P@K, nDCG, coverage, novelty, diversity, dup-F1), realtime UI, auth, encryption (AES-GCM BYO keys), CI-clean TypeScript, and a Vercel/Convex deploy.

- **Verified working:** built and exercised live in-browser — guest auth !' ranked feed of real HN/TechCrunch stories !' dashboard eval run !' settings. Production build is clean (TypeScript + lint).
- **Pragmatic tradeoffs documented:** Reddit/X rate-limit handling, the eval's proxy-relevance when feedback is sparse, and the diversity cap that the raw ranker motivates — all surfaced honestly rather than hidden.

Resources used

- Next.js 16 App Router, React 19, Tailwind v4.
- Convex (DB + auth [@convex-dev/auth](#) + scheduled functions).
- [fast-xml-parser](#) for runtime-agnostic RSS/Atom parsing inside Convex actions.
- HN Algolia API; public RSS/Atom feeds; OpenAI/Anthropic REST for optional summaries.
- The challenge brief + the referenced ChatGPT pipeline outline; the parent repo's Python [ingestion-service/deduplication-service](#) as conceptual reference.
- Built with Claude Code.

Run / deploy

See [README.md](#). TL;DR: `npm i && npx convex dev && npm run dev`; deploy with `npx convex deploy + vercel --prod` (set `NEXT_PUBLIC_CONVEX_URL`, `KEY_ENCRYPTION_SECRET`, and Convex Auth keys).